

SIMATS ENGINEERING ASSESSMENT TOOL 2

**COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY COURSE
CODE: CSA51**

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

Total Marks: 50

Annexure A

Code of Conduct:

I K . B h a v a n a (1 9 2 5 2 4 1 7 8) _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K.Bhavana

Question 1: Secure Password Storage in Web Applications

Scenario: A web application stores user passwords and needs to protect them against database breaches.

Part A — Why Storing Plaintext Passwords Is Insecure

Storing passwords in plaintext means the raw, human-readable password a user types is written directly into the database exactly as entered, with no transformation applied. This is considered one of the most serious and avoidable security failures in application design. The reasoning can be broken down step by step below.

Step 1: Understand what a database breach exposes

If an attacker gains read access to the database — through SQL injection, a stolen backup, an insider threat, or a misconfigured cloud storage bucket — every credential in that table becomes immediately visible. There is no additional barrier between the breach and full account takeover.

Step 2: Recognise the password-reuse multiplier effect

Most users reuse the same or similar passwords across multiple services. A single plaintext leak therefore does not just compromise one account; it hands attackers working credentials for the same user's email, banking, and social media accounts through automated "credential stuffing" attacks.

Step 3: Consider insider and operational risk

Plaintext storage also exposes passwords to anyone with legitimate database access — developers, database administrators, or third-party analytics tools — creating unnecessary internal risk even without an external breach.

Step 4: Note the absence of any computational barrier

Because no transformation (hashing) has been applied, an attacker who obtains the data needs to perform zero additional computation to use the credentials. This removes any time delay that could otherwise allow a company to detect the breach and force password resets before the passwords are exploited.

Summary of Risks

Risk	Explanation
Immediate compromise	No cracking effort needed — passwords are usable the instant the data is stolen.
Credential stuffing	Reused passwords let attackers break into unrelated third-party accounts.
Regulatory/legal exposure	Violates data-protection obligations (e.g., GDPR Article 32, PCI-DSS) requiring appropriate technical safeguards.
Reputational damage	Plaintext breaches attract severe public and media scrutiny, eroding customer trust.
Insider misuse	Staff with DB access can view or misuse live user credentials.

Part B — A Secure Method for Storing Passwords

The recommended approach is to never store the password itself, but instead store a salted, computationally expensive cryptographic hash of the password, generated using a purpose-built password-hashing algorithm such as Argon2id (preferred), bcrypt, or scrypt — rather than a general-purpose hash like SHA-256 used alone.

Step 1: Choose a memory-hard hashing algorithm

Argon2id is recommended because it won the Password Hashing Competition and is specifically designed to resist both GPU/ASIC brute-force cracking (via high memory cost) and side-channel timing attacks (by combining the data-dependent Argon2i and data-independent Argon2d modes).

Step 2: Generate a unique, random salt per user

A cryptographically random salt (at least 16 bytes) is generated for every account and stored alongside the hash. The salt ensures that two users with identical passwords produce completely different hash outputs, defeating precomputed "rainbow table" attacks.

Step 3: Apply the hash function with tuned cost parameters

The password and salt are combined and passed through Argon2id with tuned parameters — for example a memory cost of 64 MB, 3 iterations, and parallelism of 1 (values are tuned to take roughly 250–500 ms on production hardware). This deliberately slows down each guess, making brute-force and dictionary attacks computationally expensive even with powerful hardware.

```
// Example (Node.js, using the argon2 library)

const argon2 = require('argon2');

const hash = await argon2.hash(plainPassword, {
  type: argon2.argon2id,
  memoryCost: 65536,    // 64 MB
  timeCost: 3,
  parallelism: 1
});

// store `hash` (salt is embedded in the output string)
```

The database stores only the algorithm identifier, cost parameters, salt, and resulting hash — never the plaintext password. This entire package is normally returned as a single self-describing string by the hashing library, simplifying future verification.

Step 5: Verify logins by re-hashing, never by decrypting

On login, the system re-computes the hash of the submitted password using the stored salt and parameters, then compares it to the stored hash using a constant-time comparison function. Hashing is one-way by design, so there is no "decrypt" step — this is what makes the scheme secure even if the database is later stolen.

```
const valid = await argon2.verify(storedHash, submittedPassword);

if (valid) { /* grant access */ }
```

Step 6: Add defence-in-depth controls

- Enforce a strong password policy and check new passwords against known-breach lists (e.g., HaveIBeenPwned's k-anonymity API).
- Apply rate limiting / account lockout and CAPTCHA to slow online brute-force attempts.
- Enable multi-factor authentication (MFA) so a leaked hash alone is insufficient for account takeover.
- Optionally add a server-side "pepper" — a secret key stored outside the database (e.g., in a hardware security module or secrets manager) — combined with the password before hashing, so a stolen database alone is still insufficient without the pepper.

Justification of the Choice

Argon2id (or bcrypt/scrypt as acceptable alternatives) is justified over faster general-purpose hashes such as MD5 or SHA-256 because those algorithms are intentionally fast — a property that is desirable for data-integrity checks but catastrophic for password storage, since it allows attackers to test billions of guesses per second on modern GPUs. Memory-hard functions like Argon2id deliberately trade speed for resistance to large-scale parallel cracking, directly addressing the threat model of a stolen password database.

Algorithm	Suitability for Passwords	Reason
Plaintext	Unsuitable	No protection at all if breached.
MD5 / SHA-1 / SHA-256 (unsalted)	Unsuitable	Fast, deterministic, vulnerable to rainbow tables and GPU brute force.
SHA-256 + salt (single pass)	Weak	Salt stops rainbow tables, but still too fast against GPU cracking.
bcrypt	Good	Adaptive cost factor, industry-proven since 1999.
scrypt	Good	Memory-hard, resists ASIC/GPU cracking.
Argon2id	Best practice	Memory-hard + side-channel resistant; OWASP's current top recommendation.

Question 2: Digital Signatures for Document Authentication

Scenario: A company wants to ensure that official documents sent electronically are authentic and unaltered.

Part A — How Digital Signatures Can Be Used in This Scenario

A digital signature uses asymmetric (public-key) cryptography to bind a document to its sender in a way that is mathematically verifiable and tamper-evident. The process below walks through how the company would apply this to outgoing official documents.

Step 1: Generate a key pair

The signing party (e.g., the company's authorised officer or a document-management system acting on their behalf) generates an asymmetric key pair: a private key kept strictly secret (ideally in a hardware security module or secure enclave) and a corresponding public key that can be shared openly, typically via a digital certificate.

Step 2: Hash the document

Before signing, the document is passed through a cryptographic hash function (e.g., SHA-256), producing a fixed-length digest that uniquely represents the document's exact content. Hashing first, rather than signing the whole document directly, keeps the signature operation fast regardless of document size.

Step 3: Encrypt the hash with the private key to create the signature

The hash digest is encrypted (signed) using the sender's private key. This output is the digital signature. Because only the sender possesses the private key, only the sender could have produced this specific signature for this specific document.

```
// Conceptual example (RSA-PSS / ECDSA workflow)

digest      = SHA256(document)

signature   = Sign(privateKey, digest)

sendPackage = { document, signature, certificate }
```

Step 4: Attach the signature and certificate to the document

The signature, along with the sender's public-key certificate (issued by a trusted Certificate Authority confirming the public key genuinely belongs to the company), is bundled with the document and transmitted to the recipient — commonly embedded directly in PDF or XML document standards (e.g., PAdES, XAdES).

Step 5: Recipient verifies the signature

On receipt, the recipient's software independently re-computes the hash of the received document, decrypts the received signature using the sender's public key to recover the original hash, and compares the two hash values.

- If the two hashes match → the document is confirmed authentic and unaltered.
- If the hashes differ → the document has either been modified in transit or was not signed with the claimed private key, and verification fails.

Step 6: Validate the certificate chain

The recipient also checks that the certificate is issued by a trusted Certificate Authority, has not expired, and has not been revoked (via CRL or OCSP), confirming the public key genuinely belongs to the company and not to an impersonator.

Visual Summary of the Workflow

Stage	Sender / Recipient	Action
1	Sender	Generate private/public key pair; obtain certificate from a CA.
2	Sender	Hash the document (e.g., SHA-256).
3	Sender	Encrypt hash with private key → digital signature.
4	Sender	Send document + signature + certificate.
5	Recipient	Re-hash received document.
6	Recipient	Decrypt signature with sender's public key.
7	Recipient	Compare hashes; validate certificate chain.

Part B — Justification: Integrity and Non-Repudiation

This method satisfies the company's two core requirements — authenticity/integrity and non-repudiation — for the following reasons, explained step by step.

Step 1: How integrity is ensured

Because the signature is mathematically derived from the exact byte-content of the document via a cryptographic hash, even a single-character change to the document (a modified figure, an inserted clause, a changed date) produces a completely different hash value due to the avalanche effect of secure hash functions. Any such change causes signature verification to fail, immediately alerting the recipient that the document is not in its original, unaltered state.

Step 2: How non-repudiation is ensured

Non-repudiation means the sender cannot later credibly deny having sent or approved the document. This holds because the signature could only have been produced using the sender's private key, which — under proper key-management practice — is accessible to no one else. Combined with a trusted CA-issued certificate binding that key to the company's identity, the signature serves as strong legal and technical evidence of origin, comparable to (and in many jurisdictions legally equivalent to) a handwritten signature under electronic-signature laws such as eIDAS (EU) or the ESIGN Act (US).

Step 3: Why this is stronger than alternative controls

Simpler controls such as checksums or plain hashing alone only prove a file has not changed since the checksum was generated — they do not prove who generated it, since anyone can compute a checksum. Passwords or shared secrets can prove identity to a system but do not produce a portable, independently verifiable proof attached to the document itself. Digital signatures uniquely combine both properties — content integrity and verifiable origin — in a single cryptographic artefact.

Integrity vs Non-Repudiation at a Glance

Property	What It Guarantees	How the Signature Provides It
Integrity	The document has not been altered since signing.	Hash comparison fails on any modification.
Authenticity	The document really came from the claimed sender.	Only the sender's private key could create a valid signature.
Non-repudiation	The sender cannot deny sending/approving it.	Signature is cryptographically bound to a CA-certified private key unique to the sender.

- Store private keys in a Hardware Security Module (HSM) or secure key vault — never on a general-purpose file server.
- Use short-lived or regularly rotated certificates and enforce revocation checking (OCSP) on verification.
- Apply timestamping (RFC 3161) so signatures remain verifiable even after a certificate later expires.
- Log all signing events for audit purposes to reinforce the non-repudiation claim in legal proceedings.

Consolidated Justification

Taken together, digital signatures directly resolve the company's stated concern — verifying that official electronic documents are authentic and unaltered — by binding cryptographic proof of both content integrity and sender identity into a single, independently verifiable artefact, backed by a trusted third-party certificate authority. This makes the scheme suitable for legally and operationally sensitive document exchange, satisfying both the technical and evidentiary requirements described in the scenario.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand